

## Quiz1(Section A)

Date: 2-11-2025

Marks:20

### Q1. Multiple Choice Questions (MCQs)

1. In a singly linked list, how do you move to the next node?
  - (A) current = previous;
  - (B) current = current->next;
  - (C) current->next = current;
  - (D) current = next->current;      (E) None of Above
2. What happens when you try to access current->next->next in a singly linked list, but current->next is NULL?
  - (A) Moves to the next node safely
  - (B) Segmentation fault (runtime error)
  - (C) Loops back to the first node
  - (D) Inserts a new node automatically      (E) None of Above
3. Which of the following correctly deletes a node from the middle of a singly linked list?
  - (A) current = current->next;
  - (B) previous->next = current->next; free(current);
  - (C) free(current);
  - (D) current->next = NULL; free(current);      (E) None of Above
4. How do you move to the previous node in a doubly linked list?
  - (A) current = current->next;
  - (B) current = current->prev;
  - (C) current->prev = NULL;
  - (D) current = head;      (E) None of Above
5. In a **doubly linked list**, what happens when you delete a middle node?
  - (A) Only the next pointer is updated
  - (B) The prev and next pointers of adjacent nodes are updated
  - (C) The entire list gets deleted
  - (D) Nothing happens, as deletion is not allowed in DLL      (E) None of Above

6. If head->prev != NULL in a doubly linked list, what does it indicate?
- (A) The list is circular
  - (B) There is a loop in the list
  - (C) The list is not properly initialized
  - (D) The head is the second node      (E) None of Above
7. In a circular linked list, how do you detect the end of traversal?
- (A) When current == NULL
  - (B) When current->next == head
  - (C) When current->next == NULL
  - (D) When current->prev == head      (E) None of Above
8. How does a circular doubly linked list differ from a normal doubly linked list?
- (A) It has no NULL pointers
  - (B) It contains a tail pointer
  - (C) It cannot be traversed in reverse
  - (D) It requires less memory      (E) None of Above
9. In a circular singly linked list, how do you insert a new node at the beginning?
- (A) Set newNode->next = head and move head to newNode
  - (B) Set newNode->next = tail
  - (C) Set head = newNode and tail->next = newNode
  - (D) Both (A) and (C)      (E) None of Above
10. How do you detect a loop in a **singly linked list**?
- (A) Traverse using a slow and fast pointer technique
  - (B) Check if any node's next is pointing to an earlier node
  - (C) Count the number of nodes while traversing
  - (D) Both (A) and (B)      (E) None of Above

## Q2: Complete Code

### 1. Insert a Node at the Beginning of a Singly Linked List

```
struct Node {  
    int data;  
    Node* next; };  
  
void insertAtBeginning(Node** head, int value) {  
    Node* newNode = new Node();  
    newNode->data = value;  
    // Fill in the missing code below  
    _____;  
    _____;  
}
```

**Question:** Fill in the blanks to correctly insert the new node at the beginning of the singly linked list.

### 2. Delete a Node from a Singly Linked List

```
void deleteNode(Node** head, int key) {  
    Node* temp = *head, *prev = NULL;  
    // Case: Head node contains key  
    if (temp != NULL && temp->data == key) {  
        *head = _____; // Move head  
        delete temp;  
        return;  
    }  
    // Search for the key  
    while (temp != NULL && temp->data != key) {  
        _____;  
        temp = temp->next;  
    }  
    if (temp == NULL) return;  
    // Remove node  
    prev->next = _____;  
    delete temp; }
```

**Question:** Fill in the blanks to delete a node from a **singly linked list**.

### 3. Traverse a Doubly Linked List

```
void traverseDoublyLinkedList(Node* head) {  
  
    Node* temp = head;  
  
    while (temp != NULL) {  
  
        cout << _____ << " "; // Print data  
  
        temp = _____; // Move forward  
  
    }  
}
```

**Question:** Fill in the blanks to correctly traverse a **doubly linked list**.

### 4. Insert a Node at the End of a Circular Singly Linked List

```
void insertAtEnd(Node** head, int value) {  
  
    Node* newNode = new Node();  
  
    newNode->data = value;  
  
    newNode->next = *head;  
  
    if (*head == NULL) {  
  
        *head = newNode;  
  
        _____; // Point to itself  
  
        return;  
  
    }  
  
    Node* temp = *head;  
  
    while (temp->next != _____) {  
  
        temp = temp->next;  
  
    }  
  
    temp->next = newNode;  
  
    newNode->next = *head;  
  
}
```

**Question:** Fill in the blanks to properly insert a node at the end of a **circular singly linked list**.

## 5. Reverse a Singly Linked List

```
Node* reverseList(Node* head) {  
    Node* prev = NULL;  
    Node* current = head;  
    Node* next = NULL;  
  
    while (current != NULL) {  
        next = _____; // Save next node  
        current->next = _____; // Reverse pointer  
        prev = _____; // Move prev forward  
        current = next;  
    }  
    return prev;  
}
```

**Question:** Fill in the blanks to correctly reverse a **singly linked list**.